OPMIG-09: Troubleshooting & Validation

> **Series:** OPMIG — OpenPipeline Migration | **Notebook:** 9 of 10 | **Created:** December 2025 | **Last Updated:** 10/30/2026
> **Level:** Intermediate
> **Prerequisites:** OPMIG-01 through OPMIG-08
> **Estimated Time:** 45 minutes

Table of Contents

1. Migration Validation Checklist
2. Troubleshooting Decision Trees
3. Pipeline Health Monitoring
4. Emergency Rollback Procedures ⭐ EXPANDED
5. Performance Troubleshooting ⭐ NEW
6. Common Issues & Solutions
7. Parsing Validation
8. Volume & Cost Validation
9. Performance Optimization
10. Rollback Procedures
11. Ongoing Maintenance
12. Migration Complete Checklist
13. 🎉 Congratulations!

Learning Objectives

By the end of this notebook, you will be able to:

- ⭐ **Navigate decision trees** to diagnose common OpenPipeline issues
- **Validate pipeline processing** using DQL verification queries
- **Troubleshoot parsing failures** and identify root causes
- **Debug masking and security processing** to ensure PII/PHI protection
- ⭐ **Execute emergency rollback procedures** when issues occur
- ⭐ **Resolve performance problems** in high-volume environments
- **Identify bucket and routing issues** with systematic diagnosis
- **Test end-to-end pipeline flows** before production deployment

---

Prerequisites

Requirement	Details
Dynatrace Environment	Dynatrace SaaS with Grail and active OpenPipeline pipelines — Managed is not covered by this series
Permissions	`settings:read` (OpenPipeline configuration), `storage:logs:read`
API Access	`logs.read` token scope
Knowledge	OPMIG-01 through OPMIG-08; at least one active pipeline configured

---

Migration Validation Checklist

Use this checklist to validate your migration:

Data Flow Validation

- [ ] All log sources are flowing to OpenPipeline
- [ ] Data volumes match expectations

- [ ] No data loss detected
- [ ] Timestamps are correct

Parsing Validation

- [ ] Log levels extracted correctly
- [ ] Custom fields parsed as expected
- [ ] JSON payloads properly flattened
- [ ] No parsing errors in logs

Routing Validation

- [ ] Logs routed to correct pipelines
- [ ] Bucket assignments are correct
- [ ] No unrouted data accumulating
- [ ] Nothing still processed by the classic pipeline — see *Verify nothing still takes the classic pipeline* below

Security Validation

- [ ] Sensitive data masked properly
- [ ] No PII visible in stored logs
- [ ] Compliance requirements met

Extraction Validation

- [ ] Metrics being generated
- [ ] Events appearing correctly
- [ ] Business events available

Verify source-level coverage

> **Coverage vs. volume — the log-module self-monitoring events.** For the *All log sources are flowing* check above, verify coverage at the source level — a source OneAgent detects but does not ingest won't appear as volume at all, so a volume query alone can't distinguish it from a source that was never discovered. The log

> module emits a `log_source.status` event per discovered source into `dt.system.events`, carrying `log.source.file_status` and `log.source.ingest_status` — so it reveals sources OneAgent **detected but is not ingesting**, which a `fetch logs` count cannot show (no stored records means nothing to count). It scans events, not raw logs, and needs only event-read.
>
> This event stream is **Early Access** and requires **opt-in** (the OneAgent log module must be enabled to send self-monitoring events; its content folds into the built-in *Log ingest Overview* dashboard for Dynatrace **1.339+** — verify it is flowing in your tenant before relying on it). See **FAQ-08** (Recommended Approach) for the full file-status × ingest-status coverage matrix and the `fetch logs` fallback for tenants without the opt-in.

```
// Log-source coverage from the OneAgent log-module self-monitoring events.
// Unlike `fetch logs` (which sees only sources that produced records), this
// surfaces sources OneAgent DETECTED but is NOT ingesting. Scans events, not
raw logs.
fetch dt.system.events, from: now() - 24h
| filter event.provider == "Log Module" and event.type == "log_source.status"
| dedup event.id, sort:{ timestamp desc }
| filter event.status == "Active"
| fieldsAdd coverage =
    if(log.source.file_status == "FILE_STATUS_OK" and log.source.ingest_status
== "Ingested", "Available | Ingesting", else:
    if(log.source.file_status == "FILE_STATUS_OK" and log.source.ingest_status
== "Not ingested", "Available | NOT ingesting", else:
    if(log.source.ingest_status == "Partially ingested", "Partial ingestion",
    else: "Unsupported or issue")))
| summarize sources = count(), by:{coverage}
| sort sources desc
```

Verify nothing still takes the classic pipeline

The classic pipeline does not go away when you create your first OpenPipeline route. It stays as the **default route**, and every record that no route matches keeps going through it:

> *"Data that doesn't match is processed by the default route and continues to be processed by the classic pipeline until you turn off the rules."*

That makes "how much still takes the classic pipeline" the most direct measure of whether this migration is finished. The classic pipeline is also what the ready-made **Check your upgrade readiness** dashboard reports under *OpenPipeline adoption*. Records it processed carry the pipeline id `logs:default` (or `bizevents:default`) in `dt.openpipeline.pipelines`. That field is populated for classic records too, which is why the *Final migration validation summary* at the end of this notebook counts them separately. A plain `isNotNull(dt.openpipeline.pipelines)` check reads as fully routed even when nothing has been migrated.

Record-level check, the method the migration guide documents. This scans raw records, so keep the window short:

```
// Records still processed by the classic pipeline, last 24 hours.
// For business events: fetch bizevents and "bizevents:default".
fetch logs, from: now() - 24h
| summarize total = count(), classic = countIf(in(dt.openpipeline.pipelines,
"logs:default"))
| fieldsAdd classic_pct = round(100.0 * classic / total, decimals: 2)
```

Trend across both data types, from self-monitoring. OpenPipeline records how many records each pipeline received in the `dt.sfm.openpipeline.routing.records` metric, so you can follow the classic share over weeks without scanning a single log line. `pipeline_id == "default"` is the classic pipeline, the same identifier as `logs:default` above:

```
// Classic vs OpenPipeline record volume over 30 days, logs and business
events.
// Reads a self-monitoring metric: no raw records are scanned.
timeseries records = sum(dt.sfm.openpipeline.routing.records, default: 0),
from: now() - 30d,
  filter: { in(configuration, {"logs", "bizevents"}) },
  by: { configuration, pipeline_id }
| fieldsAdd path = if(pipeline_id == "default", "classic pipeline", else:
"OpenPipeline")
| fieldsAdd total = arraySum(records)
| summarize records = sum(total), by:{ configuration, path }
| sort configuration asc, records desc
```

On the validation tenant (09/29/2026) the two methods agreed. Logs showed **0** classic records in 24 hours out of 24.7 million, and the metric showed no classic log volume over 30 days. Business events, which had never been migrated, showed **993,927** classic records over 30 days in the metric and **993,375** by record count. The small gap is the difference between metric and record-scan timing at the window edges. **Check business**

events explicitly. They migrate separately from logs, so a tenant that has finished logs can still be sending all its business events through the classic pipeline.

Zero is necessary, not sufficient. The migration guide is explicit about what the number does not prove:

> *"Zero classic pipeline usage does not guarantee that all data is fully converted, only that it was not routed via the Classic route."*

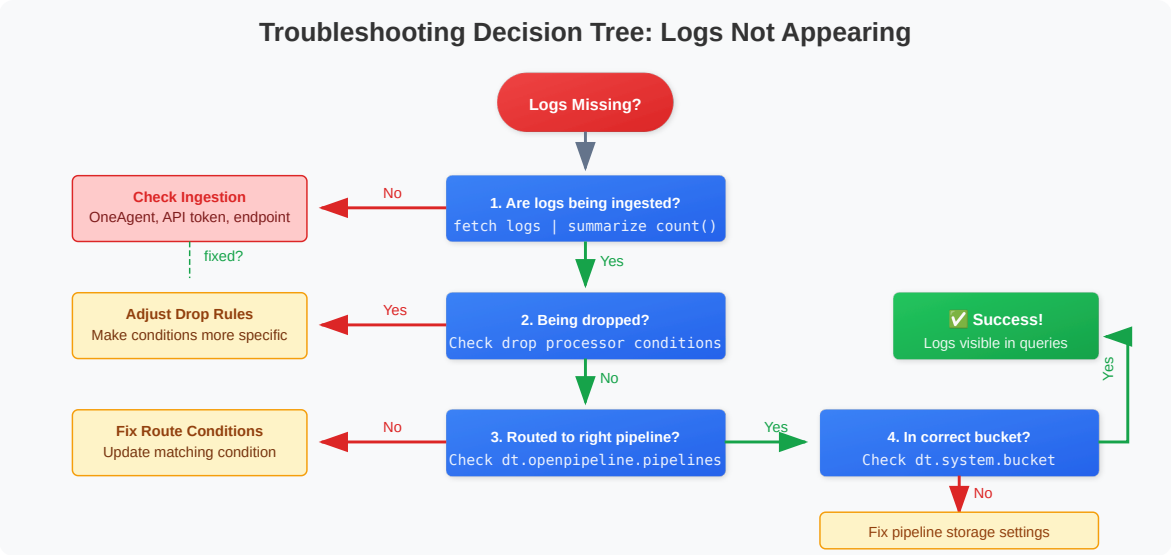
Data that arrives rarely, such as a monthly batch job or an audit source that only logs on failure, may simply not have arrived in your window. Before you turn off a classic processing rule, account for every source the rule handled, not just the ones that showed up this week.

> **Sources:** Upgrade from classic pipeline to OpenPipeline (DT docs) — the `in(dt.openpipeline.pipelines, "logs:default")` filter and both quotes above. **Dictionary:** `dt.openpipeline.pipelines` (`experimental`), read 09/29/2026. The metric's `pipeline_id` / `configuration` dimensions were read from the `metrics` command, and both queries were executed against a Dynatrace tenant on 09/29/2026 with the counts shown. **Derived:** reading `pipeline_id == "default"` as the classic pipeline rests on the two methods agreeing on that tenant, not on a docs statement about the metric.

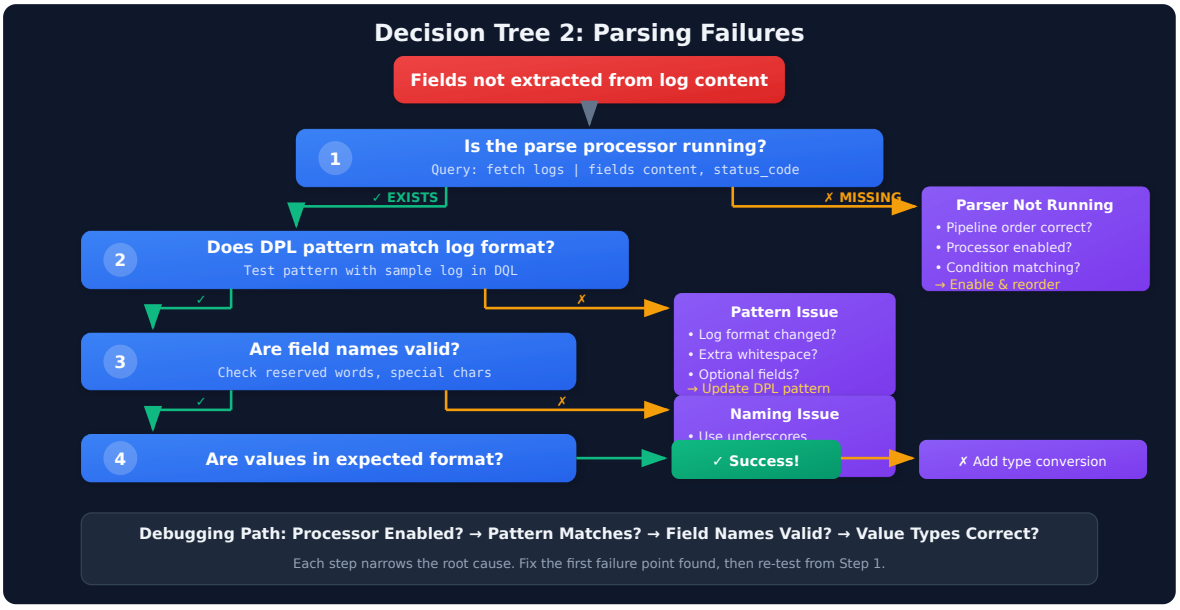
Troubleshooting Decision Trees

Visual decision trees for diagnosing and resolving common OpenPipeline issues.

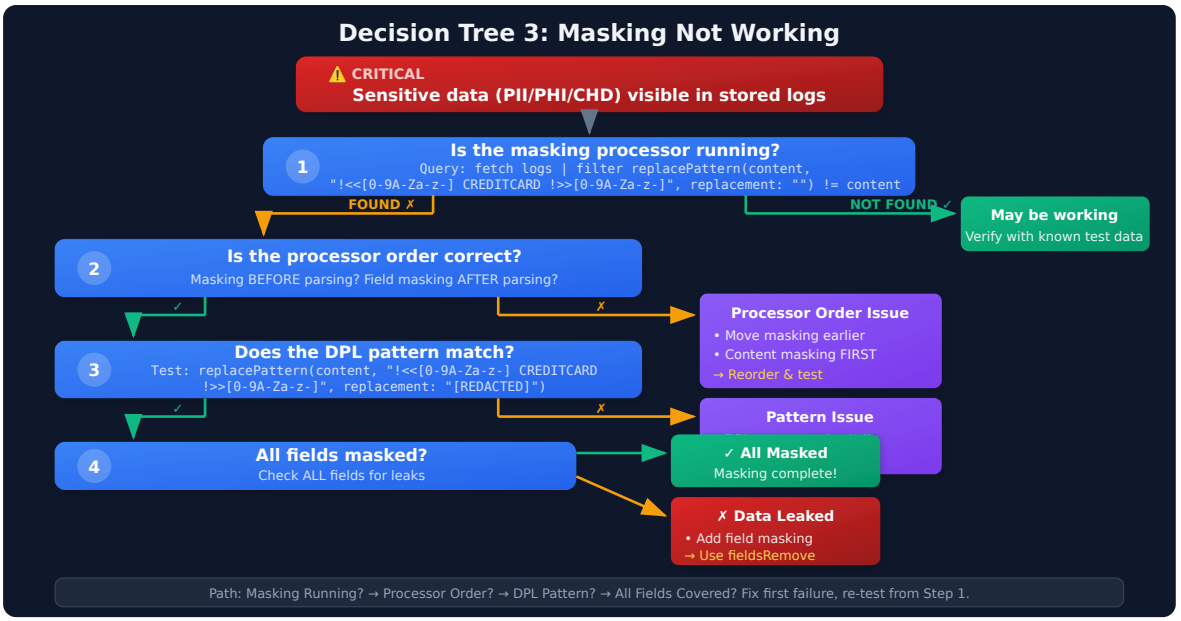
Decision Tree 1: Logs Not Appearing



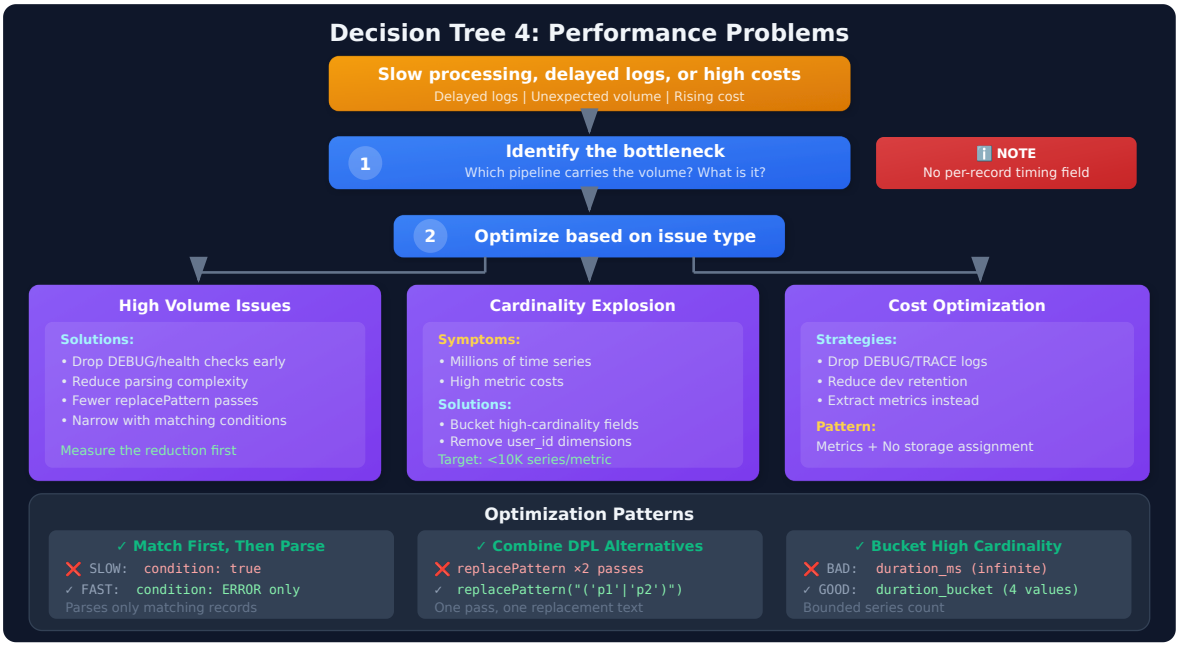
Decision Tree 2: Parsing Failures



Decision Tree 3: Masking Not Working



Decision Tree 4: Performance Problems



Pipeline Health Monitoring

Monitor the overall health of your OpenPipeline configuration.

```
// Overview: Log volume by pipeline (last 24 hours)
fetch logs, from: now() - 24h
| summarize {log_count = count()}, by: {dt.openpipeline.pipelines}
| sort log_count desc
```

```
// Pipeline volume trend over time
fetch logs, from: now() - 24h
| makeTimeseries {log_count = count()}, by: {dt.openpipeline.pipelines},
interval: 1h
```

```
// Check for logs still on the classic pipeline (potential routing issues).
// isNull() never fires: classic records carry dt.openpipeline.pipelines too
fetch logs, from: now() - 1h
| filter in(dt.openpipeline.pipelines, "logs:default")
| summarize {unrouted_count = count()}, by: {log.source}
| sort unrouted_count desc
```

```
// Bucket distribution check
fetch logs, from: now() - 24h
| summarize {
    log_count = count()
  }, by: {dt.system.bucket, dt.openpipeline.pipelines}
| sort log_count desc
```

```
// Log source health - are all expected sources sending data?
fetch logs, from: now() - 1h
| summarize {
    log_count = count(),
    last_seen = max(timestamp)
  }, by: {log.source}
| fieldsAdd minutes_since_last = (now() - last_seen) / 1m
| sort minutes_since_last desc
```

Emergency Rollback Procedures ⭐ EXPANDED

When OpenPipeline issues occur in production, follow these emergency procedures.

Scenario 1: Critical Logs Being Dropped

Symptoms:
- Production error logs missing
- Security events not visible
- Business-critical logs vanished

Immediate Actions:

1. **Disable Drop Processor (UI)**

```
Settings → Process and contextualize → OpenPipeline → Logs
→ Pipelines → select pipeline → Processing stage
→ Find drop processor → Toggle OFF
→ Save pipeline

Effect: Immediate (within 1-2 minutes)
```

2. **Disable Drop Processor (Settings API)**

Pipelines, routes and ingest sources are **Settings objects** (schemas `builtin:openpipeline.logs.pipelines`, `builtin:openpipeline.logs.routing`, `builtin:openpipeline.logs.ingest-sources`); there is no separate `/api/v2/openpipeline/...` endpoint. Each processor in a pipeline object carries an `enabled` flag.

```
   # Get the pipeline objects (token scope settings.read) — keep this file as
your backup
   curl -X GET "https://{tenant}.live.dynatrace.com/api/v2/settings/objects?
schemaIds=builtin:openpipeline.logs.pipelines" \
     -H "Authorization: Api-Token {token}" &gt; pipelines-backup-YYYYMMDD.json

   # Edit the pipeline's value: set the drop processor's "enabled" to false
   # (Processing-stage processors are under value.processing.processors[])

   # Write the whole object back (token scope settings.write); body =
{"value": { ...edited value... }}
   # Add ?validateOnly=true first to check the object without saving it
   curl -X PUT
"https://{tenant}.live.dynatrace.com/api/v2/settings/objects/{objectId}" \
     -H "Authorization: Api-Token {token}" \
     -H "Content-Type: application/json" \
     -d @updated-pipeline.json
```

3. **Verify Logs Reappear**

```
   fetch logs, from: now() - 5m
   | filter log.source == "critical-service"
   | summarize count()
```

Post-Incident:
- Review drop condition logic
- Add safeguards (never drop ERROR/FATAL)
- Test in staging before re-enabling

Scenario 2: Parsing Breaking Log Ingestion

Symptoms:
- Logs delayed or not appearing
- Pipeline processing time spiking
- Parse errors in pipeline metrics

Immediate Actions:

1. **Disable Parse Processor**

```
Settings → OpenPipeline → Select pipeline
→ Disable DQL parse processor
→ Save

Result: Logs stored unparsed (content field only)
Impact: Fields not extracted, but logs visible
```

2. **Check Processing Metrics**

```
OpenPipeline records no per-record processing time or status:
dt.openpipeline.processing_time_ms and dt.openpipeline.status are not
Grail fields (no row in dt.semantic_dictionary.fields, 10/02/2026).
Use the volume query below and the dt.sfm.openpipeline.* metrics in
Performance Troubleshooting instead.
```

Alternative Query:

```
// Monitor pipeline log volume and sources instead
// dt.openpipeline.pipelines is an array of "&lt;scope&gt;:&lt;pipeline
id&gt;" strings, so == never matches
fetch logs, from: now() - 1h
| filter matchesValue(dt.openpipeline.pipelines, "*my-pipeline*")
| summarize
    log_count = count(),
    sources = collectDistinct(dt.openpipeline.source)
```

3. **Test Parse Pattern Offline**

```
// Use makeTimeseries or static data to test pattern
data record(
  content = "192.168.1.1 - GET /api/users HTTP/1.1 200 1024"
)
| parse content, """
    IPADDR:client_ip SPACE '-' SPACE
    LD:method SPACE LD:path SPACE LD:protocol SPACE
    INT:status_code SPACE INT:bytes
  """
| fields client_ip, method, status_code
```

Post-Incident:

- Fix parse pattern
- Test with production log samples
- Re-enable processor

Scenario 3: Masking Failure (PII/PHI Exposed)

Symptoms:
- Credit card numbers visible in queries
- SSNs, patient IDs, or emails not redacted
- Compliance violation alerts

CRITICAL - Immediate Actions:

1. **STOP STORING THE EXPOSED RECORDS (if actively leaking)**

```
   A pipeline object has no single on/off switch. Fastest options, in the UI
or via the
   Settings API PUT shown in Scenario 1:
   - Fix the masking processor (see step 5), or
   - Add a Drop record processor at the top of the pipeline's Processing stage
whose
     matching condition selects the affected records, until masking is fixed.

   Do NOT just disable the route: the records then leave through the Default
route
   without this pipeline's masking.
```

2. **Identify Exposure Window**

```
   // Widen from: until first_exposure stops moving. replacePattern finds a
card number
   // anywhere in the line; matchesPhrase on a fixed number would miss every
other card.
   // The !&lt;&lt; / !&gt;&gt; bounds stop digit runs inside UUIDs being
counted as exposed cards.
   fetch logs, from: now() - 7d
   | filter log.source == "payment-service"
   | filter replacePattern(content, "!&lt;&lt;[0-9A-Za-z-] CREDITCARD
!&gt;&gt;[0-9A-Za-z-]", replacement: "") != content
   | summarize
       first_exposure = min(timestamp),
       last_exposure = max(timestamp),
       affected_count = count()
```

3. **Delete the Exposed Records**

```
   Use the Grail record deletion API (POST /delete:execute, "Grail - Storage
Record Deletion"
   in the Dynatrace API definitions) — or Sensitive Data Center, which is
built on it — with a
   DQL filter scoped to the exposure window from step 2.

   Deletion is final and can't be undone: export the evidence the incident
record needs first.
   Requires a custom policy with storage:records:delete plus the matching
storage:*:read and
   storage:buckets:read permissions.
```

4. **Restrict Read Access While Investigating**

```
   Bucket read access is granted through IAM policies (storage:buckets:read
plus table
   permissions such as storage:logs:read), and record-level access through
dt.security_context —
   there is no per-bucket "Access Control" screen. Tighten the policies bound
to the groups that
   can read the affected bucket until the exposed records are deleted.
```

> **Sources:** Record deletion in Grail via API (DT docs) — *"The record deletion API is primarily designed to help users remove selected records containing sensitive data."*; *"Record deletion is final and can't be undone."*

5. **Fix Masking and Re-Enable**

```
   // Test masking pattern before deploying
   data record(
     content = "Payment processed: card=4111-1111-1111-1111, cvv=123"
   )
   | fieldsAdd content = replacePattern(content, "DIGIT{4} '-' DIGIT{4} '-'
DIGIT{4} '-' DIGIT{4}", "[PAN_REDACTED]")
   | fieldsAdd content = replacePattern(content, "'cvv=' DIGIT{3,4}", "cvv=
[REDACTED]")
   | fields content

   // Expected: "Payment processed: card=[PAN_REDACTED], cvv=[REDACTED]"
```

Post-Incident (MANDATORY):

- [ ] Document exposure timeline

- [ ] Notify compliance/security team
- [ ] File incident report (PCI-DSS, HIPAA, GDPR)
- [ ] Review all masking patterns
- [ ] Implement automated compliance checks
- [ ] Add pre-production masking tests

Scenario 4: Routing to Wrong Bucket

Symptoms:
- Production logs in dev bucket (7-day retention)
- Audit logs in default bucket (missing compliance)
- High-value logs being dropped prematurely

Immediate Actions:

1. **Verify Current Routing**

```
   fetch logs, from: now() - 1h
   | filter log.source == "production-api"
   | summarize {count = count()}, by: {dt.system.bucket}
| sort count desc
```

2. **Create Temporary High-Retention Bucket**

```
   Settings → Storage management → Bucket storage management → Bucket
   Name: emergency_recovery_logs
   Table: logs
   Retention: 90 days
   (Access is granted by IAM policy, not on the bucket — restrict it there)
```

3. **Point the Pipeline's Bucket Assignment at the Safe Bucket**

```
   Pipeline → Bucket assignment stage
   Add a Bucket assignment processor as the FIRST processor in the stage:
     Matching condition: true
     Bucket: emergency_recovery_logs
   Save

   Effect: new records from this pipeline land in the safe bucket
   (the stage is first-match-only, so the first matching processor wins)
```

4. **Fix Routing and Bucket Assignment**

```
   A route selects a PIPELINE; the bucket is chosen by the pipeline's Bucket
assignment
   processors. Fix each in its own place:

   Dynamic routing:
     Route: log.source == "production-api"  →  pipeline: production-api

   Pipeline production-api → Bucket assignment stage (first match only):
     Processor 1: status == "ERROR"    →  bucket prod_error_logs (90 days)
     Processor 2: true                 →  bucket prod_logs (35 days)

   Then remove the temporary emergency_recovery_logs processor.
```

Post-Incident:
- Review all routing rules for correctness
- Document expected bucket for each log source
- Add monitoring alerts for unexpected bucket usage

Scenario 5: Complete Pipeline Rollback

When to Use:
- Multiple critical issues
- Unknown root cause
- Need time to investigate safely

Full Rollback Procedure:

1. **Stop Routing to the Pipeline**

```
   # Routes live in one settings object per scope (schema
builtin:openpipeline.logs.routing);
   # each entry in value.routingEntries[] has its own "enabled" flag
   curl -X GET "https://{tenant}.live.dynatrace.com/api/v2/settings/objects?
schemaIds=builtin:openpipeline.logs.routing" \
     -H "Authorization: Api-Token {token}" &gt; routing-backup-YYYYMMDD.json

   # Set "enabled": false on the route(s) whose pipelineId is the problem
pipeline,
   # then PUT the object back ({"value": { ... }})
   curl -X PUT
"https://{tenant}.live.dynatrace.com/api/v2/settings/objects/{routingObjectId}
" \
     -H "Authorization: Api-Token {token}" \
     -H "Content-Type: application/json" \
     -d @routing-updated.json

   # Records that no longer match a route leave through the Default route
```

2. **Restore Previous Version (if available)**

```
   # Restore the pipeline from the settings-object backup taken before the
change:
   # PUT its saved "value" back to the same objectId
   curl -X PUT
"https://{tenant}.live.dynatrace.com/api/v2/settings/objects/{objectId}" \
     -H "Authorization: Api-Token {token}" \
     -H "Content-Type: application/json" \
     -d @pipeline-backup-YYYYMMDD.json
```

3. **Verify Rollback Success**

```
   fetch logs, from: now() - 5m
   | summarize
       count(),
       pipelines = collectDistinct(dt.openpipeline.pipelines),
       buckets = collectDistinct(dt.system.bucket)
```

4. **Communicate Status**

```
Template:

Subject: OpenPipeline Rollback - [Pipeline Name]

Timeline:
- [HH:MM] Issue detected: [description]
- [HH:MM] Rollback initiated
- [HH:MM] Rollback complete

Impact:
- Logs affected: [time window]
- Data loss: [yes/no]
- Service affected: [list]

Next Steps:
- Root cause analysis in progress
- Fix ETA: [estimate]
- Testing in staging before re-deploy
```

Emergency Rollback Checklist

> **Sources:** Settings API - GET objects (DT docs) — *"To execute this request, you need an access token with settings.read scope."*; Settings API - PUT an object (DT docs) — *"If true, the request runs only validation of the submitted settings object, without saving it."*; OpenPipeline limits (DT docs) — *"We recommend to migrate your legacy configurations to the Settings API."*; Data flow (DT docs) — *"If no route matches the record, the record is routed via the Default route."*

The schema IDs (`builtin:openpipeline.logs.pipelines`, `builtin:openpipeline.logs.routing`, `builtin:openpipeline.logs.ingest-sources`) and the per-processor and per-route `enabled` flags used above were read from a validation tenant's settings objects on 09/28/2026 — confirm them in your own tenant with the GET before you edit anything.

Before ANY pipeline changes:
- [ ] Backup current pipeline configuration (Settings API GET of the pipeline and routing objects)
- [ ] Test in staging environment first
- [ ] Document expected behavior

- [ ] Have rollback plan ready
- [ ] Notify team of change window

During incident:
- [ ] Identify affected time window
- [ ] Disable problematic processor/pipeline
- [ ] Verify logs flowing again
- [ ] Document all actions taken
- [ ] Communicate to stakeholders

After rollback:
- [ ] Root cause analysis
- [ ] Fix and test in staging
- [ ] Peer review changes
- [ ] Document lessons learned
- [ ] Update runbooks

Performance Troubleshooting ⭐ NEW

Diagnose and resolve performance issues in high-volume OpenPipeline environments.

Performance Issue 1: Slow Processing / High Latency

Symptoms:
- Logs arriving later than expected
- One pipeline carrying far more volume than planned
- Queries timing out or slow to return

Diagnosis Queries:

OpenPipeline does not record processing time per record or per processor — there is no `dt.openpipeline.processing_time_ms`, `dt.openpipeline.processor_*` or `dt.system.ingestion_time` field (none has a row in `dt.semantic_dictionary.fields`, while `dt.openpipeline.source` and `dt.openpipeline.pipelines` do; checked 10/02/2026). Diagnose by **volume** and by **ingest warnings**.

```
// 1. Records out of each pipeline, from OpenPipeline self-monitoring — no log
scan
timeseries records_out = sum(dt.sfm.openpipeline.pipelines_out.records,
default: 0),
  from: now() - 6h, interval: 15m, by: { configuration, pipeline_id }
| fieldsAdd total = arraySum(records_out)
| sort total desc
```

```
// 2. Records Dynatrace changed or truncated on ingest, by source
fetch logs, from: now() - 1h
| filter isNotNull(dt.ingest.warnings)
| summarize n = count(), by: {dt.ingest.warnings, dt.openpipeline.source}
| sort n desc
```

To find an expensive processor, there is no timing to read: narrow its matching condition, or disable processors one at a time in a staging copy of the pipeline and compare.

> **Dictionary:** `dt.openpipeline.pipelines` (`experimental`), `dt.openpipeline.source` (`experimental`), `dt.ingest.warnings` (`experimental`); no row for `dt.openpipeline.processing_time_ms`, `dt.openpipeline.status` or `dt.system.ingestion_time` under `filter startsWith(name, "dt.openpipeline") OR startsWith(name, "dt.ingest")`, read 10/02/2026 (control: the same filter returned 10 rows). Metric keys `dt.sfm.openpipeline.pipelines_out.records` and `dt.sfm.openpipeline.not_stored.records` read from the `metrics` command the same day.

Common Causes & Solutions:

Cause	Solution
Complex DPL patterns in parse/mask	Simplify patterns; narrow the processor's matching condition
Too many processors	Consolidate logic, remove unused
High-volume debug logs	Add a drop processor early
Expensive fieldsAdd logic	Pre-compute values, use Inline lookup
Large log messages	Remove or truncate fields you do not need

In community practice these are the usual levers; the gain from each depends on your data — measure before and after with query 1.

Optimization Example:

```
// ❌ SLOW: Multiple replacePattern in sequence
| fieldsAdd content = replacePattern(content, "'pattern1'", replacement: "X")
| fieldsAdd content = replacePattern(content, "'pattern2'", replacement: "Y")
| fieldsAdd content = replacePattern(content, "'pattern3'", replacement: "Z")

// ✅ FAST: Single DPL pattern with alternatives (replacePattern takes DPL,
not regex)
| fieldsAdd content = replacePattern(content, "
('pattern1'|'pattern2'|'pattern3')", replacement: "[REDACTED]")
```

```
// ❌ SLOW: one processor that parses every record
// Matching condition: true
parse content, """complex pattern"""

// ✅ FAST: narrow with the processor's MATCHING CONDITION, not a filter
command
// (`filter` is not enabled in a DQL processor definition)
// Matching condition: matchesPhrase(content, "ERROR")
parse content, """complex pattern"""
```

Performance Issue 2: High Volume / Rate Limiting

Symptoms:
- HTTP 429 (Too Many Requests) errors
- Logs missing during peak traffic
- OneAgent warnings about rate limits

Diagnosis Queries:

```
// 1. Identify high-volume sources (by record count — dt.system.log_size_bytes
is not a Grail field)
fetch logs, from: now() - 1h
| summarize {
    log_count = count(),
    logs_per_second = count() / 3600
}, by: {log.source}
| sort log_count desc
| limit 20
```

```
// 2. Check for volume spikes (record counts)
fetch logs, from: now() - 6h
| makeTimeseries
    log_count = count(),
    by: {log.source},
    interval: 5m
```

```
// 3. Analyze droppable logs
// toDouble() matters: long / long is integer division in DQL, so without it
the percentage is 0
fetch logs, from: now() - 1h
| summarize {
    total = count(),
    debug = countIf(loglevel == "DEBUG"),
    trace = countIf(loglevel == "TRACE"),
    health = countIf(contains(content, "/health")),
    // one countIf, so a record matching two conditions is counted once
    droppable = countIf(loglevel == "DEBUG" OR loglevel == "TRACE" OR
contains(content, "/health"))
}, by: {log.source}
| fieldsAdd droppable_pct = toDouble(droppable) / toDouble(total) * 100
| filter droppable_pct &gt; 30
| sort droppable_pct desc
```

Volume Reduction Strategies:

Strategy	Use Case	Expected Reduction
Drop DEBUG/TRACE	Development, verbose apps	40-70%
Drop health checks	Microservices, K8s	20-40%
Sampling	High-volume, low-value	80-95% (with 10-20% sample)
Metric extraction	SLI metrics, counts	99%+ (logs → metrics)

Strategy	Use Case	Expected Reduction
Aggregation	Repeated messages	50-80%

Example: Aggressive Volume Reduction

```
// Drop low-value logs (matching condition of a Drop record processor)
loglevel == "DEBUG" OR
loglevel == "TRACE" OR
matchesValue(content, "*/health*") OR
matchesValue(content, "*/metrics*") OR
matchesPhrase(content, "/ping") OR
(loglevel == "INFO" AND matchesPhrase(log.source, "chatty-service"))

// Expected: 60-80% volume reduction
```

Sampling Strategy:

```
// Sample 10% of INFO logs, keep all ERROR/WARN
// status == "ERROR" also keeps SEVERE, CRITICAL, FATAL and the rest of the
error group
| filter
    status == "ERROR" OR
    status == "WARN" OR
    (loglevel == "INFO" AND random() &amp;lt; 0.1)

// Result: ~70% reduction (if 80% are INFO)
```

Performance Issue 3: Cardinality Explosion

Symptoms:

- Metric extraction creating millions of time series
- High DPS/DDU costs for metric storage
- Queries slow due to high cardinality

Diagnosis Queries:

```
// 1. Count unique dimension combinations
fetch logs, from: now() - 1h
| filter isNotNull(user_id)  // Example high-cardinality field
| summarize
    unique_users = countDistinct(user_id),
    unique_services = countDistinct(service.name),
    potential_series = countDistinct(concat(user_id, "_", service.name))
```

```
// 2. Estimate metric cardinality
fetch logs, from: now() - 24h
| summarize
    dim1_count = countDistinct(dimension1),
    dim2_count = countDistinct(dimension2),
    dim3_count = countDistinct(dimension3)
| fieldsAdd estimated_series = dim1_count * dim2_count * dim3_count

// ⚠️ WARNING: If estimated_series &gt; 100,000 → cardinality problem!
```

```
// 3. Find high-cardinality fields
// user_id, request_id, session_id and request_path are example names: replace
them with
// attributes your logs actually carry, or every count reads 0
fetch logs, from: now() - 1h
| summarize {
    user_ids = countDistinct(user_id),
    request_ids = countDistinct(request_id),
    session_ids = countDistinct(session_id),
    paths = countDistinct(request_path)
}, by: {service.name}
| sort user_ids desc
```

Cardinality Reduction Techniques:

Problem	❌ High Cardinality	✅ Low Cardinality	Reduction
User tracking	`user_id` (1M unique)	`user_segment` (5 values)	99.9995%
API paths	`/api/users/12345`	`/api/users/{id}`	99%
Durations	`duration_ms` (infinite)	`duration_bucket` (10 buckets)	99.9%+
Timestamps	`timestamp` (infinite)	`hour_of_day` (24 values)	99.9%+

Problem	❌ High Cardinality	✅ Low Cardinality	Reduction
IP addresses	`client_ip` (100K unique)	`client_country` (200 values)	99.8%

Example: Bucketing Strategy

```
// ❌ BAD: Infinite cardinality
| fieldsAdd duration_ms = response_time
// Metric extraction: log.api.response_time (dimensions: service, path,
duration_ms)
// Result: 5 services × 100 paths × 10,000 durations = 5,000,000 time series
❌

// ✅ GOOD: Bucketed cardinality
| fieldsAdd duration_bucket =
    if(response_time &lt; 100, "fast",
    else: if(response_time &lt; 500, "normal",
    else: if(response_time &lt; 2000, "slow",
    else: "very_slow")))
// Metric extraction: log.api.response_time (dimensions: service, path,
duration_bucket)
// Result: 5 services × 100 paths × 4 buckets = 2,000 time series ✅
// Reduction: 99.96%
```

Example: Path Normalization

```
// ❌ BAD: /api/users/12345, /api/users/67890, ... (infinite paths)
| fieldsAdd api_path = request_path

// ✅ GOOD: Normalize to /api/users/{id}
// replacePattern takes a DPL pattern: literals in single quotes, INT for the
numeric ID
| fieldsAdd api_path = replacePattern(request_path, "'/api/users/' INT",
replacement: "/api/users/{id}")
| fieldsAdd api_path = replacePattern(api_path, "'/api/orders/' INT",
replacement: "/api/orders/{id}")
| fieldsAdd api_path = replacePattern(api_path, "'/api/products/' [a-f0-9-]+",
replacement: "/api/products/{uuid}")
// Result: 20 normalized paths instead of 100,000+ unique paths
```

Cardinality Budget:

```
Target: &lt; 10,000 time series per metric

Good Examples:
- 5 services × 20 endpoints × 3 status_categories = 300 series ✅
- 10 services × 5 environments × 4 duration_buckets = 200 series ✅

Bad Examples:
- 5 services × 100K user_ids = 500,000 series ❌
- 10 services × 10K request_ids × 5 methods = 500,000 series ❌

Rule of Thumb:
If dimension has &gt; 1,000 unique values → BUCKET IT or REMOVE IT
```

Common Issues & Solutions

Issue 1: Logs Not Appearing

Symptoms:
- Expected logs missing from queries
- Volume lower than expected

Diagnosis:

```
// Check if logs are being dropped
// Compare volume before and after drop processors
fetch logs, from: now() - 1h
| summarize {
    total = count(),
    debug = countIf(loglevel == "DEBUG"),
    trace = countIf(loglevel == "TRACE")
  }, by: {log.source}
```

Solution:
1. Check drop processor conditions in pipeline configuration
2. Verify routing conditions aren't too restrictive
3. Confirm log source is sending data to Dynatrace
4. Check for OneAgent or log forwarder issues

Issue 2: Parsing Not Working

Symptoms:

- `loglevel` is `NONE`
- Expected parsed fields missing

Diagnosis:

```
// Find logs with parsing failures
fetch logs, from: now() - 1h
| filter loglevel == "NONE"
| fields timestamp, log.source, dt.openpipeline.pipelines, content
| limit 20
```

Solution:

1. Test DPL pattern in DPL Architect with failing log samples
2. Check for variations in log format
3. Verify processor matching condition includes these logs
4. Order processors correctly (parse before fieldsAdd that uses parsed fields)

Issue 3: Wrong Pipeline Assignment

Symptoms:

- Logs in unexpected pipelines
- Routing not working as expected

Diagnosis:

```
// Check routing - are logs going to expected pipelines?
fetch logs, from: now() - 1h
| summarize {log_count = count()}, by: {log.source, dt.openpipeline.pipelines}
| sort log.source asc
```

Solution:

1. Review dynamic routing table configuration
2. Check routing condition order (first match wins)
3. Verify matching conditions use correct fields
4. Test conditions with sample logs

Issue 4: Masking Not Applied

Symptoms:
- Sensitive data visible in logs
- Redaction placeholders missing

Diagnosis:

```
// Check if masking is working
fetch logs, from: now() - 1h
| filter contains(content, "@")  // Potential email
| filter NOT contains(content, "[EMAIL_REDACTED]")
| fields content
| limit 20
```

Solution:
1. Verify the masking DQL processor (`replacePattern`) is in the pipeline
2. Check masking processor order (first within the Processing stage)
3. Test DPL pattern matches the sensitive data format
4. Ensure masking applies to correct fields

Issue 5: Metrics Not Generated

Symptoms:
- Expected metrics missing
- Metric queries return no data

Diagnosis:

List log-extracted metrics: browse them in the Dynatrace UI (**Observe > Metrics**), or query a specific metric with `timeseries avg_value = avg(log.your_metric_name), from: now() - 24h` .

Solution:
1. Verify extraction processor configuration
2. Check matching condition selects correct logs
3. Ensure value field exists and is numeric (for value metrics)
4. Check for dimension cardinality issues

Parsing Validation

> 🔍 **Reference queries moved to OPMIG-99.** See **OPMIG-99 § 11.1 Parsing Validation** for diagnostic queries that check parsing success rates, log-level distribution, and unparsed-source identification. Run them after configuring or modifying parse processors.

Volume & Cost Validation

> 📊 **Reference queries moved to OPMIG-99.** See **OPMIG-99 § 11.2 Volume & Cost Validation** for queries that check daily log volume by bucket, drop-processor effectiveness, log-level distribution, and top-volume sources. Useful in the first week after cutover.

Performance Optimization

Tips for optimizing OpenPipeline performance.

Processor Ordering

1. **Masking** - Always first within the Processing stage (security)
2. **Drop** - Second (reduce volume early)
3. **Technology Parsers** - Before custom parsing
4. **Custom Parse** - Before fieldsAdd that uses parsed fields
5. **fieldsAdd** - After parsing, for computed fields
6. **fieldsRemove** - Last, to clean up temporary fields

Bucket assignment and metric/event extraction are not part of this list — they are later stages whose order is fixed.

Matching Condition Optimization

Do	Don't
Use indexed fields	Complex regex on content
Check field existence first	Expensive operations always
Simple equality checks	Multiple OR conditions

Pattern Efficiency

Efficient	Less Efficient
`LD? 'prefix=' NSPACE:val` — finds the key anywhere, stops at the next space	`'prefix=' LD:val` — matches only lines that *start* with `prefix=`, then runs to end of line
Specific literals	Greedy matchers
Short patterns	Long complex patterns

```
// Identify pipelines with high volume (candidates for optimization)
fetch logs, from: now() - 1h
| summarize {hourly_volume = count()}, by: {dt.openpipeline.pipelines}
| sort hourly_volume desc
```

Rollback Procedures

If issues are detected, here's how to rollback.

Rollback Options

Level	Action
Pipeline	Disable specific pipeline, use default
Processor	Disable individual processor
Routing	Reset dynamic routing to defaults

Level	Action
Full	Disable the routes to your custom pipelines so records leave through the Default route. Whether a Classic pipeline still exists behind it depends on the environment — it is not available for accounts created from September 2026

Quick Disable Steps

1. Navigate to **Settings** → **OpenPipeline** → **Logs**
2. Select the problematic pipeline
3. Toggle the pipeline off
4. Logs will route to default pipeline

Version History

If your tenant's OpenPipeline UI offers a version history for the pipeline, you can restore a previous version from there — this series has not verified that such a view exists. The dependable path is the Settings API backup described in *Scenario 5* above: take a `GET` of the pipeline and routing objects before every change and `PUT` the saved value back to roll back.

> **Sources:** Processing in OpenPipeline (DT docs) — *"For new accounts created from September 2026, classic pipeline is not available."*

Ongoing Maintenance

After migration, establish these maintenance practices.

Daily Checks

- [ ] Log volume within expected range
- [ ] No spike in unparsed logs
- [ ] Error log levels visible

Weekly Checks

- [ ] Parsing success rates stable
- [ ] Bucket distribution as expected
- [ ] New log sources properly routed

Monthly Checks

- [ ] Review masking effectiveness
- [ ] Optimize high-volume pipelines
- [ ] Clean up unused pipelines
- [ ] Document any changes

```
// Weekly health report query
// error_logs uses the normalized `status` field: status == "ERROR" also
covers SEVERE, CRITICAL,
// ALERT, FATAL and EMERGENCY, which loglevel == "ERROR" alone misses
fetch logs, from: now() - 7d
| summarize {
    total_logs = count(),
    parsed_logs = countIf(loglevel != "NONE"),
    error_logs = countIf(status == "ERROR"),
    pipelines_used = countDistinct(dt.openpipeline.pipelines),
    sources_active = countDistinct(log.source)
  }
| fieldsAdd parse_rate = round((toDouble(parsed_logs) / toDouble(total_logs))
* 100, decimals: 1)
| fieldsAdd error_rate = round((toDouble(error_logs) / toDouble(total_logs)) *
100, decimals: 2)
```

```
// Find new log sources that may need pipeline configuration
// (records on the default route: the classic pipeline, id "logs:default").
// dt.openpipeline.pipelines is an array, so compare with in(), never ==.
fetch logs, from: now() - 24h
| filter in(dt.openpipeline.pipelines, "logs:default")
| summarize {
    log_count = count(),
    first_seen = min(timestamp)
  }, by: {log.source}
| filter log_count &gt; 100
| sort log_count desc
```

Migration Complete Checklist

Final validation before declaring migration complete.

Data Quality ✅

- [ ] All log sources flowing
- [ ] Parsing rates > 95%
- [ ] Log levels correctly extracted
- [ ] Custom fields available
- [ ] Timestamps accurate

Routing ✅

- [ ] All sources routed to correct pipelines
- [ ] Bucket assignments verified
- [ ] Dynamic routing working
- [ ] No orphaned data
- [ ] Classic pipeline share at zero for logs **and** business events, sustained over a full business cycle

Security ✅

- [ ] All PII masked
- [ ] Compliance requirements met
- [ ] No sensitive data in stored logs
- [ ] Masking verified with queries

Cost ✅

- [ ] Debug/trace logs dropped
- [ ] Volumes as expected
- [ ] Bucket retention configured
- [ ] Cost projections met

Extraction ✅

- [ ] Metrics generating
- [ ] Events appearing
- [ ] Business events available
- [ ] Dynatrace Intelligence integration working

Documentation ✅

- [ ] Pipeline configuration documented
- [ ] Routing rules documented
- [ ] Masking patterns documented
- [ ] Runbooks updated

Team ✅

- [ ] Team trained on OpenPipeline
- [ ] Support procedures updated
- [ ] Escalation paths defined
- [ ] Monitoring dashboards created

```
// Final migration validation summary
fetch logs, from: now() - 24h
| summarize {
    total_logs = count(),
    with_pipeline = countIf(isNotNull(dt.openpipeline.pipelines)),
    // The classic pipeline also sets dt.openpipeline.pipelines
("logs:default"),
    // so with_pipeline alone cannot tell migrated from unmigrated records.
    classic_pipeline = countIf(in(dt.openpipeline.pipelines, "logs:default")),
    with_loglevel = countIf(loglevel != "NONE"),
    with_masking = countIf(contains(content, "REDACTED")),
    unique_sources = countDistinct(log.source),
    unique_pipelines = countDistinct(dt.openpipeline.pipelines)
  }
| fieldsAdd
    routing_health = if(with_pipeline &gt; total_logs * 0.95, "✅ Good",
else: "⚠️ Check routing"),
    classic_health = if(classic_pipeline == 0, "✅ None on classic", else:
"⚠️ Still on classic pipeline"),
    parsing_health = if(with_loglevel &gt; total_logs * 0.90, "✅ Good",
else: "⚠️ Check parsing")
```

🎉 Congratulations!

You've completed the OpenPipeline migration notebook series!

What You've Learned

Notebook	Topic
OPMIG-01	Introduction & Why Migrate
OPMIG-02	Architecture & Key Concepts
OPMIG-03	Migration Assessment & Planning
OPMIG-04	Pipeline Configuration
OPMIG-05	Routing & Buckets
OPMIG-06	Processing & Parsing

Notebook	Topic
OPMIG-07	Metric & Event Extraction
OPMIG-08	Security & Masking
OPMIG-09	Troubleshooting & Validation

Next Steps

1. **Share knowledge** - Train your team
2. **Create dashboards** - Monitor pipeline health
3. **Iterate** - Add pipelines for new use cases
4. **Optimize** - Continuously improve performance

🆕 **New Addition (March 2026):** Migration complete? Move to **OPLOGS-01: OpenPipeline Fundamentals** for day-to-day log operations, or **OPIPE-01: OpenPipeline as a Multi-Scope Platform** for spans and metrics.

References

- OpenPipeline Documentation
- OpenPipeline Best Practices
- DQL Reference
- OpenPipeline Limits — Concrete numeric limits
- Dynatrace Community

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.